

Memoria del Proyecto Pay2Peer

1. Portada

- **Título:** Pay2Peer — Plataforma de transferencias y monedero
- **Autor/es:** Alejo Viñeta
- **Fecha:** 14/12/25
- **Versión:** 0.1

2. Resumen ejecutivo

- Este proyecto es una aplicación web de cartera y transferencia de dinero, estilo PayPal, con la estructura lógica de despliegue necesaria.
- El objetivo es crear un entorno seguro en el que tener una cartera de dinero y poder transferirlo entre carteras de forma segura, además de crear una infraestructura lógica donde alojarla.
- El objetivo es conseguir una aplicación que cumpla con los estándares legales de la Unión Europea sobre seguridad y privacidad en una aplicación financiera, con la infraestructura lógica necesaria.

3. Índice

- 1. Portada
- 2. Resumen ejecutivo
- 3. Índice
- 4. Introducción
- 5. Objetivos
- 6. Arquitectura del sistema
 - Visión general
 - Componentes principales
 - Flujo de pagos
- 7. Conclusiones
- 8. Trabajo futuro

4. Introducción

- Este proyecto se inició para el curso de ASIR con especialización en ciberseguridad, como proyecto de fin de ciclo. La idea del proyecto viene de un proyecto de aprendizaje de React realizado anteriormente, en el que solo se creó un MVP con la funcionalidad básica para una demo. Es un

proyecto que hasta ahora no he podido desarrollar más y este ciclo me ha dado la oportunidad de hacerlo, complementando mis conocimientos de desarrollo web con conocimientos de sistemas, redes y ciberseguridad para poder llevar este proyecto a término.

- El proyecto se divide en dos fases. La primera se entrega el 18/12/25 y tiene como alcance un MVP de la aplicación web. La segunda fase finaliza a finales de abril de 2026. Su alcance incluye asegurar la aplicación según estándares europeos, preparar la infraestructura lógica necesaria para desplegar este proyecto y dejarlo listo para producción.

5. Objetivos

- **Objetivo general:** construir una aplicación segura y conforme a los requisitos legales para gestionar saldos y transferencias.
- **Objetivos específicos:** autenticación, persistencia de sesión, integración con Stripe, trazabilidad de transacciones y pruebas E2E.

6. Arquitectura del sistema

Visión general

Cliente: Next.js — Backend: Node.js/Express — BD: MongoDB — Servicios: Stripe.

Frontend

- En Frontend se ha usado Next.js como framework principal. En un principio se iba a usar React, pero los cambios que ha habido durante los últimos años en React y mi mayor familiaridad con Next.js recientemente me hicieron cambiar a este último nada más empezar el proyecto. Next.js es un framework de React.
- Para CSS se ha usado Tailwind debido a la incomodidad de tener que gestionar múltiples archivos CSS, especialmente usando un framework basado en componentes, que son muy propensos a errores de CSS debido a cascadas de imports y ofuscando el CSS.
- Se ha utilizado también Recharts para las gráficas del dashboard, varias librerías adicionales de React, en particular react-stripe-js para incluir una pasarela de pago de tarjetas como método de introducir dinero a la app.
- Se ha usado jsonwebtoken para gestionar el token de autenticación que tiene encriptado el ID del usuario necesario para las llamadas a la API.

Backend

- Express.js se ha escogido como framework de backend debido a mi familiaridad con el mismo y se ha construido una API. Para los logs de las llamadas a la API se ha instalado Morgan, y Mongoose para conectar con

la base de datos de Mongo. Dotenv se encarga de leer las variables de entorno que no deben ser públicas, como API keys, de un archivo .env.

- Nodemon y CORS se han usado para facilitar el desarrollo: uno para poder hacer cambios en caliente y probarlos de inmediato, y el otro para usar una política no restrictiva de CORS en desarrollo.
- Se ha usado bcrypt para encriptar las contraseñas con el algoritmo Eks-Blowfish.
- Para proteger los endpoints que requieren autenticación se ha usado express-jwt y para limpiar y validar las requests express-validator.
- Stripe se ha usado para procesar los pagos a la aplicación a través de tarjetas y se ha usado currency.js para formatos de moneda. Se trabaja en céntimos para evitar errores de redondeo.
- SendGrid se ha usado como plataforma de mailing para emails de bienvenida al registrarse y emails de restaurar contraseña.
- RabbitMQ se ha usado para gestionar colas. Estas colas son un middleware que gestiona peticiones a la API y las guarda en caso de que no pueda procesarlas la API, para evitar que haya pérdidas de información.

Base de datos

- Se ha usado Mongo debido a su flexibilidad para un desarrollo rápido, ya que aún no tengo claro qué estructuras serán necesarias para asegurar la web y hacer cambios en una base de datos SQL es mucho más complicado. Una vez haya un diseño definitivo para la base de datos, se migrará a PostgreSQL.
-

7. Conclusiones

- Se han alcanzado los objetivos de la primera fase, pero por restricciones de tiempo debidas a imprevistos no se ha podido implementar la seguridad ni la base de datos con una visión a futuro, por lo que será necesario refactorizar muchas partes en la segunda fase. Además, existe un bug errático en el formulario proporcionado por Stripe que no se ha conseguido depurar a tiempo para la primera fase.
- Ha habido una investigación sobre cómo asegurar y desplegar correctamente el proyecto, por lo que se ha producido un aprendizaje en ese aspecto. Sin embargo, la primera fase del proyecto es meramente la infraestructura necesaria para poder empezar la parte más difícil del proyecto: la securización y la creación de la infraestructura de despliegue.

8. Trabajo futuro

- **Mejoras prioritarias:**
 - Testing.
 - Depuración del formulario de Stripe.

- **Nuevas funcionalidades planificadas:**
 - Implementación de blockchain en transacciones por seguridad.
 - Refactorización de la API para no exponer IDs de usuarios y carteras.
 - Uso de OAuth o SSO para el login con el fin de asegurarlo.
 - Dockerización del proyecto.
 - Investigar e implementar nuevas medidas de seguridad acordes a los estándares de la Unión Europea.